

AI Operations (AIOps)

Module 3 Assignment — Infrastructure & Containerization

TA Evaluation Edition — includes model solutions, detailed rubrics, and grading guidance. Do NOT distribute this document to students; use Module3_Assignment_Student.docx for that.

Total Marks	Weight	Submission
40	8% of course grade	Git repository link (Dockerfiles + Compose file + YAML + evidence) + a 2-page PDF write-up

Grading Philosophy for This Redesign

- Reward evidence over assertion throughout: a correct-looking manifest with no kubectl output, or a caching claim with no timing evidence, should not receive full marks even if the underlying file is syntactically valid.
- For Q1 and Q3 especially, watch for submissions that are clearly adapted from the lecture's hp-search-worker or predictor_app.py with only cosmetic renaming — the SHAPE of a correct Dockerfile is naturally similar across applications, but check that CONTENT (dependencies, entry point, exposed contract) is genuinely specific to the spam-detection API and, for Q3, the CSV validation workload.
- Q2 and Q4 are the two questions with no lecture-provided template at all — weight independent problem-solving accordingly and be more lenient on stylistic variation (e.g., different but equally valid Compose networking configurations, different but equally valid readiness-probe designs) as long as the underlying mechanism is demonstrably correct.

The Scenario (Reference — Same as Student Document)

All four questions build around a spam-detection REST API (TF-IDF + Naive Bayes, deliberately simple ML so Docker/Kubernetes skill is the actual assessment target), trained on a 1,000-row deterministic synthetic dataset (`random.seed(42)`) provided in full in the student document. The required contract is POST /predict (JSON in: {"text": ...}, JSON out: {"label": "spam" | "ham"}) and GET /healthz.

Verified reference numbers

The TA-reference implementation of this dataset + `TfidfVectorizer` + `MultinomialNB` reaches 100% accuracy on a held-out split — this is EXPECTED and not a red flag in student submissions; the dataset is templated and intentionally easy to classify, since the assignment is not testing modeling skill.

Question 1 — Single-Stage vs. Multi-Stage Docker (10 marks)

Model Solution

```
# --- Naive (single-stage) ---
FROM python:3.10
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8080"]

# --- Multi-stage ---
FROM python:3.10 AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --target=/build/deps -r requirements.txt

FROM python:3.10-slim
WORKDIR /app
COPY --from=builder /build/deps /usr/local/lib/python3.10/site-packages
COPY . .
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8080"]
```

Expected size reduction pattern: python:3.10 (full, ~1GB) vs. python:3.10-slim (~150MB) as the runtime base, plus exclusion of pip's build cache and any compiler toolchains pulled in transitively by scikit-learn's build dependencies — typically a 60–75% reduction, consistent with what students should have observed in Docker Lecture.

Evaluation Rubric & Grading Guidance

Criterion	Marks	Grading Notes
Naive Dockerfile builds/runs; serves /predict and /healthz correctly; size reported	3	Deduct 1 mark if /healthz is missing or always returns 200 even before the model loads (defeats its purpose as a readiness signal, relevant again in Q4)
Multi-stage Dockerfile builds/runs with identical behavior; size + % reduction reported	4	Deduct if the final stage still uses a non-slim base image (a common miss that erases most of the size benefit)
Explanation names THIS image's specific excluded artifacts, not a generic definition	3	A generic “multi-stage builds remove build tools” answer with no reference to their own base image choice or dependencies earns 1–2, not 3

Question 2 — Multi-Container Orchestration with Docker Compose (10 marks)

Model Solution

```
# app.py -- caching logic added to the predict endpoint
import os, json, hashlib
import redis

cache = redis.Redis(host=os.environ.get("CACHE_HOST", "cache"), port=6379, decode_responses=True)

def cache_key(text):
    return "pred:" + hashlib.sha256(text.encode()).hexdigest()

@app.post("/predict")
def predict(request: PredictRequest):
    key = cache_key(request.text)
    cached = cache.get(key)
    if cached is not None:
        return {"label": json.loads(cached), "cache": "hit"}

    label = model.predict([request.text])[0]
    cache.set(key, json.dumps(label), ex=3600) # 1-hour TTL
    return {"label": label, "cache": "miss"}

# docker-compose.yml
services:
  api:
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - "8080:8080"
    environment:
      - CACHE_HOST=cache
    depends_on:
      - cache
  cache:
    image: redis:7-alpine
    ports:
      - "6379:6379"
```

Verified caching mechanism (tested against a Redis-compatible in-memory server during development of this assignment): first request for a given text takes ~50ms (real inference); an identical repeated request returns in well under 1ms from cache — a 300x+ speedup is typical and should NOT be treated as suspiciously large; it is the expected order of magnitude for skipping model inference entirely.

Evaluation Rubric & Grading Guidance

Criterion	Marks	Grading Notes
Cache check/write correctly implemented; predictions remain correct on hits and misses	4	Deduct 1–2 if the cache key doesn't account for exact text matching correctly (e.g., missing

Criterion	Marks	Grading Notes
		normalization causes silent cache misses that still function but never actually hit) — note in feedback rather than penalizing heavily if predictions are still correct
docker-compose.yml correctly wires both services; api reaches cache via service name as hostname	3	Full marks even if the student uses environment: differently (e.g., a full REDIS_URL string) as long as it resolves via the service name, not a hardcoded IP
Concrete hit-vs-miss timing/log evidence provided	2	Accept any reasonable evidence format (timestamps in logs, a small timing script, even a screen recording description) — do not require a specific format, only that MISS was measurably slower than HIT
Correct Compose-vs-Kubernetes scope distinction	1	Look for: Compose = single host, multiple containers; Kubernetes = many hosts (nodes), scheduling/self-healing/scaling across them

Note to TA: this question has no lecture-provided template at all, so expect (and accept) more implementation variety than Q1/Q3/Q4 — grade the underlying mechanism, not adherence to this exact code shape.

Question 3 — Kubernetes Indexed Job: Parallel Data Validation (10 marks)

Model Solution

With 4 total allocatable CPUs and cpu: "900m" requested per pod (matching the Kubernetes Lecture convention of ~1 core per worker), at most 4 pods can run concurrently regardless of a higher parallelism value — the well-justified answer is parallelism: 4 (not higher, since it would simply queue past the CPU ceiling; not lower, since it would leave allocatable CPU idle).

```
apiVersion: batch/v1
kind: Job
metadata:
  name: signup-validation-job
spec:
  completions: 8
  parallelism: 4
  completionMode: Indexed
  backoffLimit: 4
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: validator
        image: signup-validator:latest
        env:
        - name: POD_NAME
          valueFrom: {fieldRef: {fieldPath: metadata.name}}
        - name: NODE_NAME
          valueFrom: {fieldRef: {fieldPath: spec.nodeName}}
      resources:
        requests: {cpu: "900m", memory: "256Mi"}
        limits: {cpu: "1000m", memory: "512Mi"}
```

Part 4 (6-CPU re-justification): a student might reasonably propose parallelism: 6 given 6 allocatable CPUs. However, since completions: 8, both parallelism: 4 and parallelism: 6 produce the SAME number of scheduling waves ($\text{ceil}(8/4) = 2$ and $\text{ceil}(8/6) = 2$) — raising parallelism only reduces wall-clock time here to the extent the two waves become more evenly sized (4+4 vs. 6+2), not because the wave COUNT changes. The strongest answers recognize that any parallelism at or above completions (i.e. ≥ 8) collapses everything into a single wave, and that this is the more meaningful threshold to reason about than simply “more CPUs available.”

Evaluation Rubric & Grading Guidance

Criterion	Marks	Grading Notes
Parallelism justified via an actual CPU-request-vs-availability calculation	3	parallelism: 4 with the stated reasoning is the expected answer; accept parallelism values other than 4 ONLY if the student's CPU request per pod is adjusted consistently and the arithmetic is correct
Valid Indexed Job manifest (completions: 8, correct	2	Standard deductions as in Module

Criterion	Marks	Grading Notes
restartPolicy, requests=limits, both Downward API fields)		2/3 rubrics elsewhere in this course
kubect! get pods -o wide evidence of the chosen parallelism actually being reached	2	Screenshot or pasted terminal output required — a claim with no evidence earns 0 here even if the manifest is correct
Results correctly collected via the Kubernetes API with a correct storage-provisioner justification	2	Look for: minikube's default storage provisioner is node-local / binds a PV to one node, which breaks access from pods scheduled elsewhere
6-CPU re-justification shows real reasoning (not just "more CPUs = higher parallelism")	1	Full marks requires recognizing that parallelism: 4 and parallelism: 6 both yield 2 scheduling waves given completions: 8 — the benefit is wave evenness, not wave count, until parallelism reaches 8

Question 4 — Kubernetes Deployments: Self-Healing & Rolling Updates (10 marks)

Model Solution

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: spam-api
spec:
  replicas: 2
  selector:
    matchLabels: {app: spam-api}
  template:
    metadata:
      labels: {app: spam-api}
    spec:
      containers:
        - name: api
          image: spam-api:v1
          resources:
            requests: {cpu: "250m", memory: "256Mi"}
            limits: {cpu: "500m", memory: "512Mi"}
          readinessProbe:
            httpGet: {path: /healthz, port: 8080}
            initialDelaySeconds: 3
            periodSeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: spam-api-svc
spec:
  type: NodePort
  selector: {app: spam-api}
  ports:
    - {port: 80, targetPort: 8080, nodePort: 30081}

$ kubectl delete pod <one-of-the-two-pod-names>
$ kubectl get pods -w
# a NEW pod with a new random suffix appears within seconds, replacing the deleted one

$ kubectl set image deployment/spam-api api=spam-api:v2
$ kubectl rollout status deployment/spam-api
$ kubectl rollout history deployment/spam-api
```

The self-healing mechanism: the Deployment's ReplicaSet continuously watches its actual Pod count against spec.replicas; deleting a Pod does not change spec.replicas, so the ReplicaSet controller immediately schedules a replacement to restore the desired count — this is a reconciliation loop, the same core Kubernetes principle from the Infrastructure-as-Code section of Kubernetes Lecture, now observed operating live.

Job-vs-Deployment contrast for part 3: had this been a Job instead, deleting a pod mid-run (or the pod completing successfully) would NOT trigger automatic replacement in the same way — a Job's controller tracks completions toward a fixed target, not a continuously-maintained replica count, so there is no equivalent “selfhealing to restore capacity” behavior for a finished or manually-removed Job pod.

Evaluation Rubric & Grading Guidance

Criterion	Marks	Grading Notes
Valid Deployment (2 replicas, resources, readinessProbe) and Service, both applied successfully	3	Deduct 1 if the Service selector doesn't match the Deployment's Pod template labels (a common copy-paste error)
Real self-healing evidence + correct controller explanation	3	Full marks requires naming the ReplicaSet (not just “Kubernetes” vaguely) and describing the reconcile-to-desired-count mechanism
Real rolling-update evidence (rollout status/history output) + correct Job contrast	3	The Job contrast is the differentiator from a superficial “it updates” answer — require the specific point that Jobs track completions, not standing replica count
Final Deployment-vs-Job discussion ties behavior to intended workload shape	1	Look for explicit language connecting long-running/serving to Deployments and run-to-completion/batch to Jobs

Overall Rubric Summary

Question	Marks	Skill Assessed
Q1	10	Multi-stage Docker builds and image size optimization, applied to a new application
Q2	10	Multi-container orchestration with Docker Compose (self-directed application of new material)
Q3	10	Kubernetes Indexed Jobs: design justification and empirical verification for a new batch workload
Q4	10	Kubernetes Deployments: self-healing and rolling updates
Total	40	

General TA Coordination Notes

- Grade Q2 and Q4 slightly more holistically than Q1 and Q3 — they have no lecture-provided structural template, so implementation diversity is expected and should not be penalized as long as the underlying mechanism (caching, self-healing, rolling update) is genuinely demonstrated with evidence.
- Across all four questions, the single most common way to lose marks should be MISSING EVIDENCE (no kubectl output, no timing numbers, no screenshots) rather than incorrect YAML/code — flag this pattern early in grading and consider a brief reminder to the class if it's widespread.

- If a submission's Q1/Q3 content is suspiciously close to the Kubernetes Lecture `hp-search-worker` or `predictor_app.py` structure with only names changed, check whether the SPECIFIC contract (POST `/predict`, GET `/healthz`, the CSV-shard validation logic) was actually implemented and adapted, or merely renamed — the former is full credit, the latter should lose marks under each question's specificity-focused criterion.